

Informe de Examen Práctico — Unidad 1

Generalidades de los Sistemas Distribuidos
Prototipo: Registro Distribuido

Aplicaciones Distribuidas (ISR-701) · Regular 2025-2026 · Ingeniería de Software
— UTEQ

Freddy Vladimir Farinango Guandinango

1. Introducción

El presente informe documenta el desarrollo del prototipo RegistroDistribuido, implementado como parte del Examen Práctico de la Unidad 1 de la asignatura Aplicaciones Distribuidas (ISR-701) de la Carrera de Ingeniería de Software de la Universidad Técnica Estatal de Quevedo.

El sistema simula un servicio de registro de operaciones distribuido compuesto por tres nodos réplica (N1, N2, N3) que se comunican mediante sockets TCP. La solución demuestra de forma funcional los mecanismos fundamentales estudiados en la Unidad 1: comunicación entre procesos, sincronización mediante relojes lógicos de Lamport, tolerancia a fallos mediante heartbeats, coordinación con el algoritmo Bully y autenticación por token.

El prototipo fue desarrollado en Java 21 con Maven, ejecutándose en tres procesos concurrentes sobre puertos distintos de la misma máquina, siguiendo las restricciones técnicas establecidas en el enunciado del examen.

2. Planteamiento del Problema

Una empresa de logística requiere un servicio de registro de operaciones denominado RegistroDistribuido. El servicio se compone de tres nodos réplica (N1, N2, N3) que reciben operaciones como registros de movimientos provenientes de uno o varios clientes. Por razones de disponibilidad, los nodos deben coordinarse entre sí, mantener un orden coherente de los eventos que registran y seguir funcionando aunque alguno de ellos deje de responder.

Los requerimientos funcionales del prototipo son:

Parte	Componente	Descripción
A	Comunicación TCP	Sockets TCP cliente-nodo y nodo-nodo con delimitación de mensajes
B	Relojes de Lamport	Implementación de reglas de reloj lógico y orden total de eventos
C	Tolerancia a fallos	Heartbeats periódicos y detección de nodos caídos
D	Coordinación Bully	Elección automática de coordinador y reelección ante caída del líder
E	Seguridad	Autenticación mediante token; rechazo de solicitudes sin token válido

3. Arquitectura del Sistema

El prototipo está organizado en tres paquetes principales: cliente, modelo y nodo. Cada paquete agrupa las clases según su responsabilidad dentro del sistema distribuido.

Paquete	Clase	Responsabilidad
cliente	ClienteTCP	Envía operaciones autenticadas al nodo N1 mediante TCP
modelo	RelojLamport	Implementa las reglas del reloj lógico de Lamport
modelo	EventoLog	Modela un evento con timestamp Lamport, nodo e descripción; implementa orden total
modelo	InfoNodo	Almacena nombre, puerto y estado activo/-caído de cada nodo
nodo	NodoServidor	Servidor TCP principal: valida token, aplica Lamport, replica y ordena eventos
nodo	NodoCliente	Clase auxiliar para comunicación nodo-nodo por TCP
nodo	HeartbeatManager	Hilo periódico que envía latidos y detecta nodos caídos
nodo	BullyManager	Elige al nodo activo de mayor ID como coordinador
nodo	NodoN1, NodoN2, NodoN3	Puntos de entrada que inician cada nodo en su puerto correspondiente

3.1 Puertos y roles

Nodo	Puerto	Rol por defecto
N1	5000	Receptor de clientes / Replicador
N2	5001	Réplica secundaria
N3	5002	Réplica / Coordinador Bully

4. Implementación de los Requerimientos

4.1 Parte A - Comunicación entre Procesos (TCP)

La comunicación entre el cliente y los nodos, así como entre nodos, se implementa mediante sockets TCP en Java. La clase `NodoServidor` crea un `ServerSocket` en el puerto asignado y atiende conexiones entrantes de forma iterativa. La clase `ClienteTCP` establece conexiones hacia el puerto 5000 (N1) y envía operaciones autenticadas.

La delimitación de mensajes se garantiza usando `PrintWriter.println()` en el lado emisor y `BufferedReader.readLine()` en el lado receptor. Este mecanismo usa el salto de línea como delimitador natural, asegurando que dos operaciones enviadas consecutivamente no se solapen ni se trunquen en el receptor.

El protocolo de comunicación establece el siguiente orden de líneas en cada mensaje: (1) token de autenticación, (2) timestamp Lamport (si aplica, en formato numérico), y (3)

contenido del mensaje. El servidor diferencia automáticamente si la segunda línea es un número —mensaje entre nodos— o texto —mensaje de cliente.

4.2 Parte B — Relojes Lógicos de Lamport

La clase `RelojLamport` implementa las tres reglas del reloj lógico de Lamport con sincronización de hilos (`synchronized`):

- **Evento local:** incrementa el contador en 1 (`eventoLocal()`).
- **Recepción de mensaje:** fija el reloj en $\max(\text{local}, \text{recibido}) + 1$ (`recibirMensaje(int)`).
- **Consulta:** devuelve el valor actual sin modificarlo (`getTiempo()`).

La clase `EventoLog` implementa la interfaz `Comparable` para producir un orden total consistente: primero por valor Lamport ascendente y, en caso de empate, por identificador de nodo (orden lexicográfico), cumpliendo el criterio de desempate exigido en la rúbrica. Cada vez que `N1` recibe una operación, el log se ordena con `Collections.sort()` y se imprime completo en consola.

4.3 Parte C — Tolerancia a Fallos (Heartbeats)

`HeartbeatManager` lanza un hilo en segundo plano que, cada 3 segundos, intenta enviar un mensaje `HEARTBEAT` a cada nodo vecino usando `NodoCliente.enviarMensaje()`. Si la conexión falla, el nodo objetivo es marcado como caído (`setActivo(false)`) y se registra el evento en el log. El sistema continúa operando con los nodos restantes sin intervención manual.

Si un nodo previamente marcado como caído vuelve a responder, se marca nuevamente como activo (`setActivo(true)`) y se registra la recuperación. Ambos eventos disparan inmediatamente una nueva elección de coordinador.

4.4 Parte D — Coordinación: Algoritmo Bully

`BullyManager` implementa el criterio central del algoritmo Bully: el nodo activo con el identificador más alto se convierte en coordinador. El método `elegirCoordinador()` recorre la lista de nodos, filtra los activos y selecciona al de mayor nombre (comparación lexicográfica, donde $N3 > N2 > N1$).

La elección se dispara automáticamente en tres momentos: al iniciar el sistema, cuando se detecta la caída de un nodo y cuando un nodo caído se recupera. Esto garantiza que ante la caída del coordinador, el sistema realiza una nueva elección sin intervención manual, cumpliendo el requisito de la Parte D.

4.5 Parte E — Seguridad: Autenticación por Token

Cada conexión entrante al `NodoServidor` debe iniciar con la línea `TOKEN123`. El servidor verifica este valor antes de procesar cualquier operación. Si el token es inválido o está ausente, el nodo responde con el mensaje `ERROR - Token invalido`, registra el intento en el log con Lamport y cierra la conexión. Solo las solicitudes con el token correcto son procesadas.

Este mecanismo se aplica tanto a los mensajes del ClienteTCP como a los mensajes internodo enviados por NodoCliente, asegurando que toda la comunicación del sistema esté autenticada.

5. Código Fuente del Proyecto

En esta sección se presenta el código fuente desarrollado para el prototipo RegistroDistribuido. Cada clase cumple una función específica dentro del sistema distribuido, permitiendo la comunicación TCP, sincronización mediante Lamport, detección de fallos, elección de coordinador y autenticación por token.

Código 1. ClienteTCP.java

Cliente encargado de enviar operaciones autenticadas hacia un nodo TCP.

```
package cliente;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
public class ClienteTCP {
    public static void main(String[] args) {
        try {
            for (int i = 1; i <= 3; i++) {
                Socket socket = new Socket("localhost", 5000);
                PrintWriter salida = new PrintWriter(
                    socket.getOutputStream(), true);
                BufferedReader entrada = new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()));
                salida.println("TOKEN123");
                salida.println("REGISTRO -00" + i);
                String respuesta = entrada.readLine();
                System.out.println(respuesta);
                socket.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Listing 1: ClienteTCP.java

Código 2. NodoServidor.java

Servidor principal que recibe mensajes, valida token y registra eventos con reloj Lamport.

```
package nodo;
import modelo.EventoLog;
import modelo.InfoNodo;
import modelo.RelojLamport;
import java.io.BufferedReader;
```

```

import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
public class NodoServidor {
    private static final String TOKEN_VALIDO = "TOKEN123";
    public static void iniciar(String nombreNodo, int puerto) {
        RelojLamport reloj = new RelojLamport();
        List<EventoLog> logEventos = new ArrayList<>();
        List<InfoNodo> nodos = Arrays.asList(
            new InfoNodo("N1", 5000),
            new InfoNodo("N2", 5001),
            new InfoNodo("N3", 5002)
        );
        try {
            ServerSocket servidor = new ServerSocket(puerto);
            System.out.println(nombreNodo + " escuchando en
                puerto " + puerto + "...");
            HeartbeatManager.iniciarHeartbeats(nombreNodo, nodos)
                ;
            while (true) {
                Socket cliente = servidor.accept();
                BufferedReader entrada = new BufferedReader(
                    new InputStreamReader(cliente.getInputStream()
                        ()));
                PrintWriter salida = new PrintWriter(
                    cliente.getOutputStream(), true);
                String token = entrada.readLine();
                String segundaLinea = entrada.readLine();
                int lamportRecibido = 0;
                String mensaje;
                if (segundaLinea.matches("\\d+")) {
                    lamportRecibido = Integer.parseInt(
                        segundaLinea);
                    mensaje = entrada.readLine();
                    reloj.recibirMensaje(lamportRecibido);
                } else {
                    mensaje = segundaLinea;
                }
                if (!TOKEN_VALIDO.equals(token)) {
                    int tiempo = reloj.eventoLocal();
                    EventoLog evento = new EventoLog(
                        tiempo, nombreNodo, "Token invalido");
                    logEventos.add(evento);
                    System.out.println(evento);
                    salida.println("ERROR - Token invalido");
                    cliente.close();
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```

        continue;
    }
    int tiempo = reloj.eventoLocal();
    if (mensaje.startsWith("HEARTBEAT")) {
        EventoLog eventoHeartbeat = new EventoLog(
            tiempo, nombreNodo, mensaje);
        logEventos.add(eventoHeartbeat);
        System.out.println(eventoHeartbeat);
        salida.println("OK - Heartbeat recibido por "
            + nombreNodo);
        cliente.close();
        continue;
    }
    EventoLog evento = new EventoLog(
        tiempo, nombreNodo,
        "Operacion recibida: " + mensaje);
    logEventos.add(evento);
    System.out.println(evento);
    if (nombreNodo.equals("N1")
        && !mensaje.startsWith("REPLICA")) {
        NodoCliente.enviarMensaje(
            "localhost", 5001,
            "REPLICA desde N1: " + mensaje, tiempo);
        NodoCliente.enviarMensaje(
            "localhost", 5002,
            "REPLICA desde N1: " + mensaje, tiempo);
        int tiempoReplica = reloj.eventoLocal();
        EventoLog eventoReplica = new EventoLog(
            tiempoReplica, nombreNodo,
            "Operacion replicada a N2 y N3");
        logEventos.add(eventoReplica);
        System.out.println(eventoReplica);
    }
    Collections.sort(logEventos);
    System.out.println("----- LOG ORDENADO " +
        nombreNodo + " -----");
    for (EventoLog ev : logEventos) {
        System.out.println(ev);
    }
    System.out.println("
    -----");
    salida.println("OK - " + nombreNodo
        + " registro la operacion con Lamport=" +
        tiempo);
    cliente.close();
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Listing 2: NodoServidor.java

Código 3. NodoCliente.java

Clase auxiliar para que un nodo se comuniqué con otro nodo mediante TCP.

```
package nodo;
import java.io.PrintWriter;
import java.net.Socket;
public class NodoCliente {
    public static boolean enviarMensaje(String host,
        int puerto, String mensaje, int lamport) {
        try {
            Socket socket = new Socket(host, puerto);
            PrintWriter salida =
                new PrintWriter(socket.getOutputStream(), true);
            salida.println("TOKEN123");
            salida.println(lamport);
            salida.println(mensaje);
            socket.close();
            return true;
        } catch (Exception e) {
            return false;
        }
    }
}
```

Listing 3: NodoCliente.java

Código 4. NodoN1.java

Clase principal que inicia el nodo N1 en el puerto 5000.

```
package nodo;
public class NodoN1 {
    public static void main(String[] args) {
        NodoServidor.iniciar("N1", 5000);
    }
}
```

Listing 4: NodoN1.java

Código 5. NodoN2.java

Clase principal que inicia el nodo N2 en el puerto 5001.

```
package nodo;
public class NodoN2 {
    public static void main(String[] args) {
        NodoServidor.iniciar("N2", 5001);
    }
}
```

Listing 5: NodoN2.java

Código 6. NodoN3.java

Clase principal que inicia el nodo N3 en el puerto 5002.

```
package nodo;
public class NodoN3 {
    public static void main(String[] args) {
        NodoServidor.iniciar("N3", 5002);
    }
}
```

Listing 6: NodoN3.java

Código 7. RelojLamport.java

Clase que implementa las reglas del reloj lógico de Lamport con sincronización de hilos.

```
package modelo;
public class RelojLamport {
    private int tiempo;
    public RelojLamport() {
        this.tiempo = 0;
    }
    public synchronized int eventoLocal() {
        tiempo++;
        return tiempo;
    }
    public synchronized int recibirMensaje(int tiempoRecibido) {
        tiempo = Math.max(tiempo, tiempoRecibido) + 1;
        return tiempo;
    }
    public synchronized int getTiempo() {
        return tiempo;
    }
}
```

Listing 7: RelojLamport.java

Código 8. EventoLog.java

Modelo utilizado para registrar y ordenar eventos del sistema con criterio de desempate por ID de nodo.

```
package modelo;
public class EventoLog implements Comparable<EventoLog> {
    private int lamport;
    private String nodo;
    private String descripcion;
    public EventoLog(int lamport, String nodo, String descripcion) {
        this.lamport = lamport;
        this.nodo = nodo;
        this.descripcion = descripcion;
    }
    @Override
```

```

    public int compareTo(EventoLog otro) {
        if (this.lamport != otro.lamport) {
            return Integer.compare(this.lamport, otro.lamport);
        }
        return this.nodo.compareTo(otro.nodo);
    }
    @Override
    public String toString() {
        return "[L=" + lamport + "][" + nodo + "] " + descripcion
            ;
    }
}

```

Listing 8: EventoLog.java

Código 9. InfoNodo.java

Modelo que almacena información básica (nombre, puerto, estado) de cada nodo distribuido.

```

package modelo;
public class InfoNodo {
    private String nombre;
    private int puerto;
    private boolean activo;
    public InfoNodo(String nombre, int puerto) {
        this.nombre = nombre;
        this.puerto = puerto;
        this.activo = true;
    }
    public String getNombre() { return nombre; }
    public int getPuerto() { return puerto; }
    public boolean isActive() { return activo; }
    public void setActivo(boolean activo) { this.activo = activo; }
}

```

Listing 9: InfoNodo.java

Código 10. HeartbeatManager.java

Gestiona el envío periódico de latidos y la detección de nodos caídos en tiempo acotado.

```

package nodo;
import modelo.InfoNodo;
import java.util.List;
public class HeartbeatManager {
    public static void iniciarHeartbeats(String nombreNodoActual,
        List<InfoNodo> nodos) {
        Thread hilo = new Thread(() -> {
            BullyManager.elegirCoordinador(nombreNodoActual,
                nodos);
            while (true) {
                for (InfoNodo nodo : nodos) {

```

```

        if (!nodo.getNombre().equals(nombreNodoActual)) {
            boolean respondio = NodoCliente.
                enviarMensaje(
                    "localhost", nodo.getPuerto(),
                    "HEARTBEAT desde " + nombreNodoActual
                        , 0);
            if (!respondio && nodo.isActivo()) {
                nodo.setActivo(false);
                System.out.println("[ " +
                    nombreNodoActual + " ] "
                        + nodo.getNombre() + " detectado
                            como CAIDO");
                BullyManager.elegirCoordinador(
                    nombreNodoActual, nodos);
            }
            if (respondio && !nodo.isActivo()) {
                nodo.setActivo(true);
                System.out.println("[ " +
                    nombreNodoActual + " ] "
                        + nodo.getNombre() + " volvio a
                            estar ACTIVO");
                BullyManager.elegirCoordinador(
                    nombreNodoActual, nodos);
            }
        }
    }
    try {
        Thread.sleep(3000);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
});
hilo.start();
}
}

```

Listing 10: HeartbeatManager.java

Código 11. BullyManager.java

Realiza la elección automática del coordinador activo usando el criterio del algoritmo Bully.

```

package nodo;
import modelo.InfoNodo;
import java.util.List;
public class BullyManager {
    public static void elegirCoordinador(String nombreNodoActual,
        List<InfoNodo> nodos) {
        InfoNodo coordinador = null;
        for (InfoNodo nodo : nodos) {

```

```

        if (nodo.isActivo()) {
            if (coordinador == null
                || nodo.getNombre().compareTo(
                    coordinador.getNombre()) > 0) {
                coordinador = nodo;
            }
        }
    }
    if (coordinador != null) {
        System.out.println "[" + nombreNodoActual
            + "] Coordinador actual: " + coordinador.
                getNombre());
    }
}
}

```

Listing 11: BullyManager.java

6. Análisis y Defensa Técnica

6.1 Teorema CAP — Propiedad Privilegiada

Ante una interrupción de comunicación entre dos nodos, la implementación privilegia la **Disponibilidad (Availability)** sobre la Consistencia (Consistency). Cuando un nodo deja de responder, los demás continúan procesando y registrando operaciones sin detenerse. Esto es observable directamente en el comportamiento del prototipo: Heartbeat-Manager marca al nodo caído como inactivo y el sistema sigue operando con los nodos restantes sin rechazar nuevas solicitudes.

La consecuencia es una relajación de la consistencia durante la partición: N2 y N3 pueden registrar eventos que N1 no recibe (o viceversa), generando divergencia temporal en los logs. El sistema sacrifica consistencia fuerte a cambio de mantener el servicio disponible, lo que es coherente con el teorema CAP: en presencia de partición de red (P), no es posible garantizar simultáneamente A y C; este prototipo elige A.

6.2 Falacias de la Computación Distribuida Consideradas

Durante el diseño del prototipo se debieron considerar las siguientes falacias:

- **La red es confiable:** falsa. Un nodo puede dejar de responder en cualquier momento sin previo aviso. Por ello se implementaron timeouts implícitos en las conexiones TCP y el mecanismo de heartbeats para detectar fallos.
- **La latencia es cero:** falsa. Los mensajes requieren tiempo para llegar. El intervalo de 3 segundos entre heartbeats incorpora un margen razonable para la latencia de red local, evitando falsos positivos en la detección de fallos.
- **La topología no cambia:** falsa. Los nodos pueden caer y recuperarse. `InfoNodo.setActivo()` permite actualizar dinámicamente el estado de cada nodo.
- **El transporte no tiene costo:** falsa. Cada heartbeat genera tráfico adicional. Se eligió un intervalo de 3 segundos como balance entre rapidez de detección y consumo

de recursos.

- **Los mensajes no se solapan ni truncan:** falsa. Para mitigar esto se usó `readLine()/println()` como delimitador de mensajes basado en salto de línea, garantizando que cada operación se procese de forma independiente y completa.

6.3 Tipos de Transparencia

Tipo de Transparencia	¿Se ofrece?	Justificación
Acceso	Sí	Clientes y nodos usan el mismo mecanismo TCP independientemente del interlocutor
Ubicación	Parcial	Los puertos son fijos y conocidos; no hay directorio de nombres dinámico
Fallos	Parcial	Se detecta la caída y se continúa operando, pero el cliente puede notar el fallo si el nodo destino es N1
Replicación	Sí	N1 replica automáticamente hacia N2 y N3 sin que el cliente lo solicite explícitamente

6.4 Propuesta de SLA de Disponibilidad

Se propone un SLA de disponibilidad del 99.9% (tres nueves) para el sistema RegistroDistribuido, considerando que opera con tres réplicas y tolerancia a un fallo parcial.

Parámetro	Valor
SLA propuesto	99.9%
Tiempo total anual	525,600 minutos ($365 \times 24 \times 60$)
Tiempo de inactividad admisible	525.6 minutos $\approx$ 8.76 horas/año
Inactividad mensual máxima	$\approx$ 43.8 minutos/mes

Este nivel es alcanzable con tres nodos réplica y recuperación automática mediante heartbeats. Para alcanzar 99.99% (cuatro nueves, $\approx$ 52.6 minutos/año) sería necesario incorporar persistencia en disco y consenso tipo Raft.

6.5 Bully vs. Consenso Raft

Bully (implementado)	Raft (alternativa)
Simple de implementar	Mayor tolerancia a fallos simultáneos
Elección rápida en red pequeña	Consenso distribuido robusto (quorum)
No garantiza consistencia del log	Replicación consistente del estado del log
Vulnerable a particiones de red	Maneja correctamente split-brain
Bajo costo de mensajes	Mayor número de mensajes (elección + AppendEntries)
Adecuado para prototipo académico	Recomendado para sistemas en producción

Reemplazar Bully por Raft introduciría las fases de RequestVote, AppendEntries y la gestión del término (term), aumentando la complejidad del código y el tráfico de mensajes,

pero garantizando que el log replicado sea consistente incluso ante particiones de red, lo que el algoritmo Bully no puede asegurar.

7. Conclusiones

El prototipo RegistroDistribuido permitió implementar y verificar en la práctica los principales mecanismos estudiados en la Unidad 1 de Aplicaciones Distribuidas:

- La comunicación TCP con delimitación por líneas garantizó la integridad de los mensajes entre cliente y nodos, y entre nodos entre sí.
- Los relojes lógicos de Lamport produjeron un orden total consistente de eventos, con desempate por identificador de nodo, cumpliendo el criterio de causalidad del sistema.
- El mecanismo de heartbeats permitió detectar la caída de un nodo en un tiempo máximo de 3 segundos y continuar operando con los nodos restantes sin intervención del operador.
- El algoritmo Bully garantizó la elección automática y dinámica del coordinador, tanto en el arranque del sistema como ante la caída del líder actual.
- La autenticación mediante token rechazó de forma correcta las solicitudes no autorizadas, registrando el intento en el log distribuido.

El análisis del teorema CAP reveló que la implementación privilegia la disponibilidad sobre la consistencia en escenarios de partición, lo cual es coherente con el objetivo de mantener el servicio activo con los nodos disponibles. Las falacias de la computación distribuida, particularmente la confiabilidad de la red y la latencia cero, fueron consideradas en el diseño de los timeouts y el protocolo de delimitación de mensajes.

Como trabajo futuro, la sustitución del algoritmo Bully por consenso tipo Raft permitiría garantizar la consistencia del log replicado y alcanzar niveles de SLA superiores (99.99%) en entornos de producción.

Anexos: Evidencias de Ejecución del Prototipo

En esta sección se presentan las capturas de pantalla que evidencian el funcionamiento del prototipo desarrollado. Las imágenes muestran la ejecución de los nodos, comunicación TCP, autenticación, relojes Lamport, replicación, heartbeats, detección de fallos y elección de coordinador.

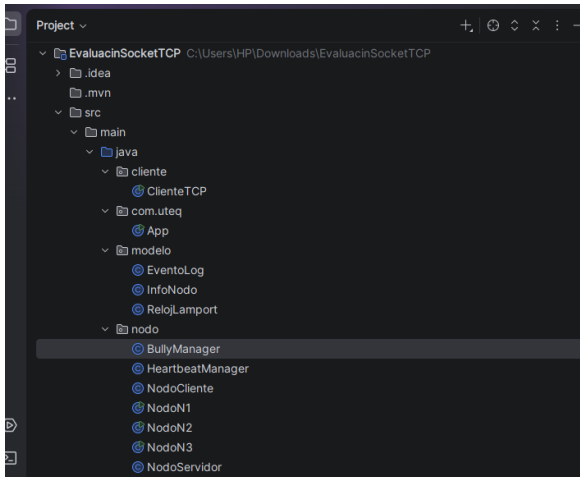


Figure 1: *Árbol de carpetas y clases del proyecto en IntelliJ IDEA.*

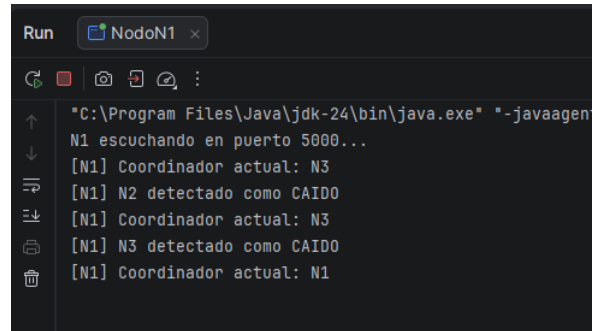


Figure 2: *Nodo N1 escuchando en el puerto 5000 e iniciando heartbeats.*

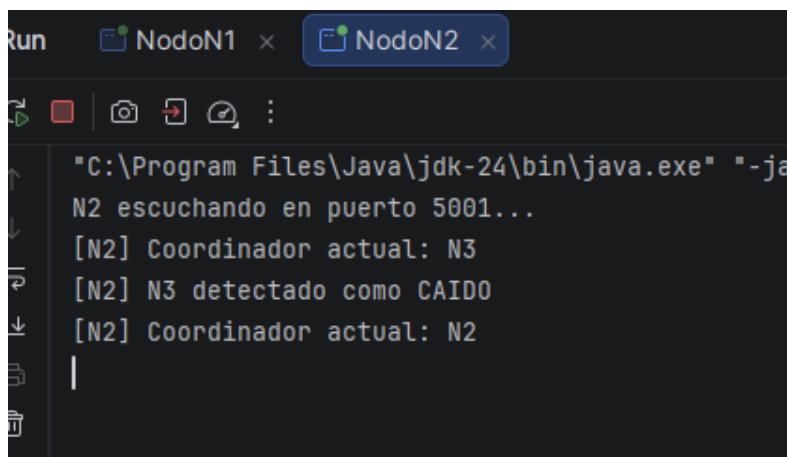


Figure 3: *Nodo N2 escuchando en el puerto 5001 e iniciando heartbeats.*

```

Run  [NodoN1] x [NodoN2] x [NodoN3] x
C:\Program Files\Java\jdk-24\bin\java.exe "-java
N3 escuchando en puerto 5002...
[N3] Coordinador actual: N3
[L=2][N3] HEARTBEAT desde N1
[L=4][N3] HEARTBEAT desde N2
[L=6][N3] HEARTBEAT desde N1
[L=8][N3] HEARTBEAT desde N2
[L=10][N3] HEARTBEAT desde N1
[L=12][N3] HEARTBEAT desde N2
[L=14][N3] HEARTBEAT desde N1

```

Figure 4: *Nodo N3 escuchando en el puerto 5002 e iniciando heartbeats.*

```

OK - N1 registro la operacion con Lamport=91
OK - N1 registro la operacion con Lamport=93
OK - N1 registro la operacion con Lamport=95

```

Figure 5: *Mensaje OK recibido por el cliente tras el registro de la operación.*

```

Run  [NodoN1] x [NodoN2] x [NodoN3] x [ClienteTCP] x
OK - N1 registro la operacion con Lamport=91
OK - N1 registro la operacion con Lamport=93
OK - N1 registro la operacion con Lamport=95
Process finished with exit code 0

```

Figure 6: *ClienteTCP enviando las tres operaciones autenticadas al nodo N1.*

```

Run  [NodoN1] x [NodoN2] x [NodoN3] x
[L=88][N1] HEARTBEAT desde N3
[L=90][N1] HEARTBEAT desde N2
[L=91][N1] Operacion recibida: REGISTRO-001
[L=92][N1] Operacion replicada a N2 y N3
[L=93][N1] Operacion recibida: REGISTRO-002
[L=94][N1] Operacion replicada a N2 y N3
[L=95][N1] Operacion recibida: REGISTRO-003
[L=96][N1] Operacion replicada a N2 y N3

```

Figure 7: *Nodo N1 mostrando el evento registrado con el valor del reloj Lamport.*

```

Run  [NodoN1] x [NodoN2] x [NodoN3] x [ClienteTCP] x
[L=82][N2] HEARTBEAT desde N3
[L=84][N2] HEARTBEAT desde N1
[L=86][N2] HEARTBEAT desde N3
[L=88][N2] HEARTBEAT desde N1
[L=93][N2] Operacion recibida: REPLICA desde N1: REGISTRO-001
[L=95][N2] Operacion recibida: REPLICA desde N1: REGISTRO-002

```

Figure 8: *Nodo N2 recibiendo la réplica del mensaje enviado desde N1.*

```

Run  [NodoN1] x [NodoN2] x [NodoN3] x [ClienteTCP] x
[L=72][N3] HEARTBEAT desde N2
[L=74][N3] HEARTBEAT desde N1
[L=76][N3] HEARTBEAT desde N2
[L=93][N3] Operacion recibida: REPLICA desde N1: REGISTRO-001
[L=95][N3] Operacion recibida: REPLICA desde N1: REGISTRO-002
[L=97][N3] Operacion recibida: REPLICA desde N1: REGISTRO-003
[L=99][N3] HEARTBEAT desde N1
[L=101][N3] HEARTBEAT desde N2

```

Figure 9: *Nodo N3 recibiendo la réplica del mensaje enviado desde N1.*

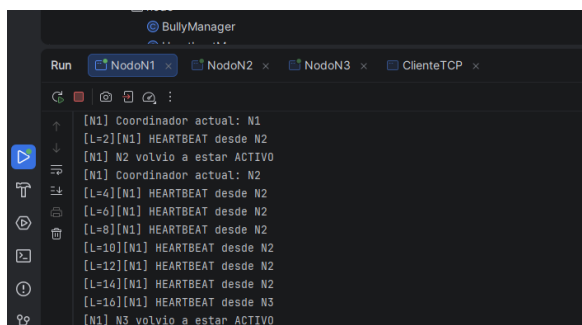


Figure 10: *Log ordenado completo mostrado en consola tras la recepción de operaciones.*

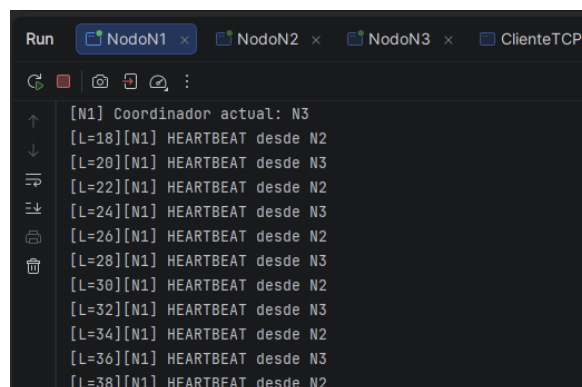


Figure 11: *Mensajes HEARTBEAT intercambiados periódicamente entre los nodos.*

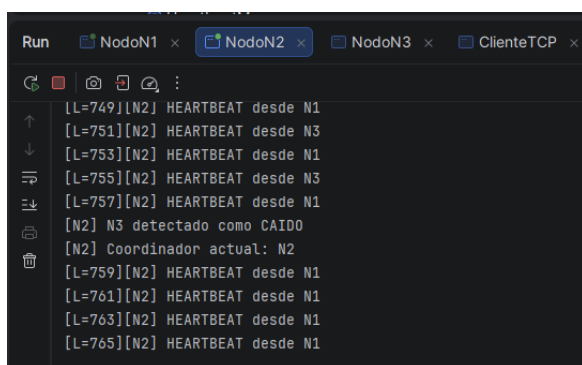


Figure 12: *N1 o N2 detectando a N3 como caído tras la interrupción de heartbeats.*



Figure 13: *Salida en consola mostrando ‘Coordinador actual: N2’ tras la caída de N3.*

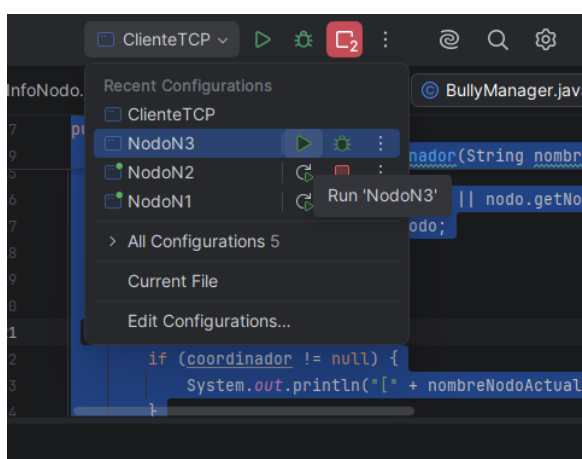


Figure 14: *N3* vuelve a estar activo y los demás nodos registran su recuperación.

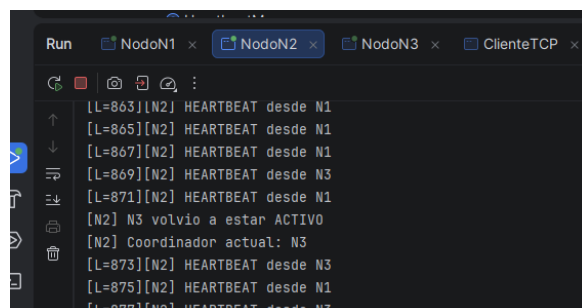


Figure 15: *Tras la recuperación de $N3$, la elección vuelve a asignarle el rol de coordinador.*

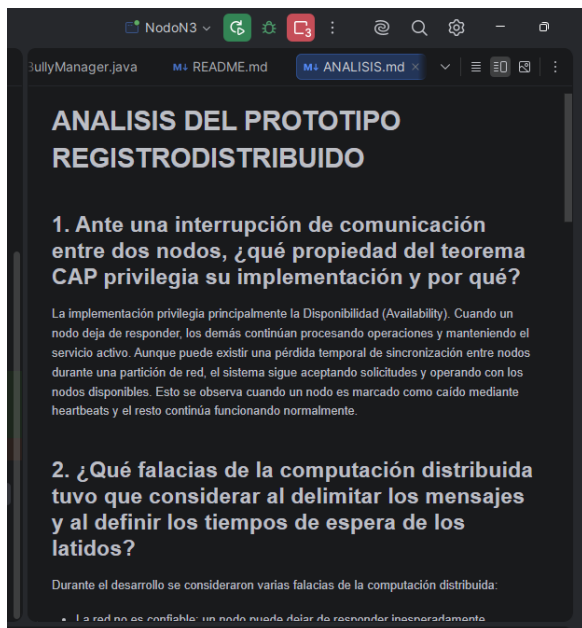


Figure 16: Archivo *ANALISIS.md* con las respuestas al componente de análisis técnico.

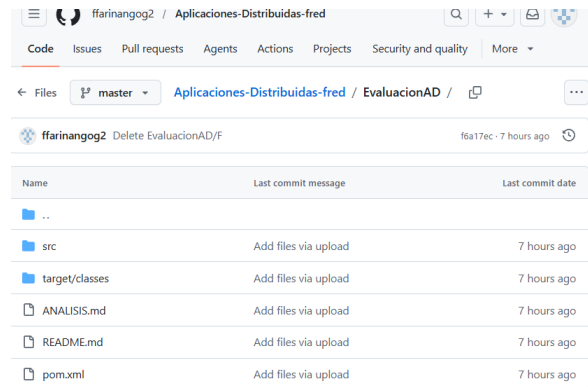


Figure 17: Repositorio *Git* con historial de commits y estructura completa del proyecto.

Revisión de Intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	FARINANGO GUANDINANGO FREDDY VLADIMIR
Comenzado el	Martes, 2 de Junio de 2026, 07:57
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:07
Tiempo empleado	0:09:50.947934
Calificación obtenida	16.00 / 20.00
Calificación final	8.00 / 10.00

PREGUNTA 1 *Finalizado* 1.00 / 1.00

Un carrito de compras prioriza la disponibilidad (AP) y emplea consistencia eventual. Un usuario agrega un ítem y, al recargar de inmediato, no lo ve; segundos después sí aparece. ¿Cómo se explica esto correctamente?

Seleccione una alternativa:

- a. Es el comportamiento esperado: alta consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata devuelve datos actualizados.
- ✓ b. **Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.**
- c. Es un fallo del sistema que viola el teorema CAP.
- d. El sistema cambió automáticamente a un modo CP e indica que el sistema no es tolerante a particiones.

La respuesta correcta es: Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.

PREGUNTA 2 *Finalizado* 1.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- a. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.
- ✓ b. **Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron “sí” pueden quedar bloqueados indefinidamente esperando la decisión final.**
- c. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron “sí” quedan libres del bloqueo indefinidamente esperando la

decisión final.

- d. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción.

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA 3*Finalizado***1.00 / 1.00**

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

- a. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer.
- ✓ b. **Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.**
- c. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.
- d. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

PREGUNTA 4*Finalizado***1.00 / 1.00**

En el algoritmo Bully (del matón), un nodo con identificador 3 detecta la caída del coordinador y lanza una elección enviando mensajes a los nodos de identificador mayor (4, 5 y 6). Los nodos 5 y 6 están caídos, pero el nodo 4 responde. ¿Quién será el coordinador final?

Seleccione una alternativa:

- a. El nodo 3, porque fue quien inició la elección.
- b. El nodo 6, porque es el de mayor identificador entre los nodos vivos.
- ✓ c. **El nodo 4, porque es el de mayor identificador entre los nodos vivos.**
- d. Ninguno, porque existen nodos caídos durante la elección.

La respuesta correcta es: El nodo 4, porque es el de mayor identificador entre los nodos vivos.

PREGUNTA 5*Finalizado***1.00 / 1.00**

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?

Seleccione una alternativa:

- ✓ a. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.
- b. El sistema distribuido siempre procesa más rápido.
- c. El sistema distribuido no admite fallos parciales.
- d. El sistema paralelo no puede tener múltiples procesadores.

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

PREGUNTA 6

Finalizado

1.00 / 1.00

Un acuerdo de nivel de servicio (SLA) promete una disponibilidad anual del 99,9 %. ¿Aproximadamente cuánto tiempo de inactividad permite al año, y qué implicaría elevarlo al 99,99 %?

Seleccione una alternativa:

- a. ≈ 1 minuto al año; el 99,99 % lo duplicaría.
- b. ≈ 88 horas al año; el 99,99 % lo empeoraría.
- c. $\approx 3,65$ días al año; el 99,99 % no cambiaría nada.
- ✓ d. $\approx 8,76$ horas al año; al 99,99 % se reduciría a $\approx 52,6$ minutos al año.

La respuesta correcta es: $\approx 8,76$ horas al año; al 99,99 % se reduciría a $\approx 52,6$ minutos al año.

PREGUNTA 7

Finalizado

1.00 / 1.00

Un sistema replicado requiere un quórum de mayoría (más de la mitad de los nodos) para confirmar una operación y mantener la consistencia. Con 5 réplicas, ¿cuántos fallos por caída simultáneos puede tolerar sin perder la capacidad de formar quórum?

Seleccione una alternativa:

- ✓ a. 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas.
- b. 3, porque con 2 réplicas vivas todavía hay mayoría.
- c. 0, porque cualquier caída rompe el quórum.
- d. 4, porque basta con que una réplica siga viva.

La respuesta correcta es: 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas.

PREGUNTA 8*Finalizado***1.00 / 1.00**

Un equipo necesita invocar métodos sobre objetos remotos entre dos máquinas virtuales de Java, pasando objetos como parámetros y manejando automáticamente la serialización. ¿Qué mecanismo es el más apropiado y por qué?

Seleccione una alternativa:

- a. TCP, porque garantiza la entrega ordenada y fiable de los objetos.
- ✓ b. **Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.**
- c. Sockets TCP crudos, porque ofrecen el mayor control de bajo nivel.
- d. Llamada a procedimiento remoto clásica estilo C, porque soporta orientación a objetos de forma nativa.

La respuesta correcta es: Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.

PREGUNTA 9*Finalizado***1.00 / 1.00**

¿Cuál enunciado distingue correctamente la “tolerancia a fallos” de la “prevención de fallos”?

Seleccione una alternativa:

- a. La tolerancia a fallos elimina por completo todos los fallos posibles, mientras que la prevención pone en alerta al administrador sobre posibles fallos.
- b. Son términos sinónimos.
- c. La prevención de fallos solo es aplicable al hardware, mientras que la tolerancia a fallos es tanto al software como al hardware.
- ✓ d. **La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.**

La respuesta correcta es: La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.

PREGUNTA 10*Finalizado***1.00 / 1.00**

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

- ✓ a. **Transparencia de migración.**

- b. Transparencia de acceso.
- c. Transparencia de ubicación.
- d. Transparencia de reubicación.

La respuesta correcta es: Transparencia de migración.

PREGUNTA 11

Finalizado

1.00 / 1.00

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- a. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales.
- b. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ✓ c. **La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.**
- d. La integridad de los datos en tránsito.

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

PREGUNTA 12

Finalizado

0.00 / 1.00

Relacione la opción correcta:

Concepto	Respuesta marcada
Sockets TCP	(no emparejado correctamente)
Invocación remota de métodos (RMI)	Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs.
Middleware orientado a mensajes (colas)	Comunicación asíncrona y desacoplada, donde emisor y receptor no necesitan coincidir en el tiempo.
Sockets UDP	Datagramas sin garantía de entrega ni orden, a cambio de menor latencia.

La respuesta correcta es: Sockets UDP → Flujo de bytes fiable y ordenado que no preserva los límites entre mensajes. , Invocación remota de métodos (RMI) → Datagramas sin garantía de entrega ni orden, a cambio de menor latencia. , Middleware orientado a mensajes (colas) → Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs.

PREGUNTA 13

Finalizado

1.00 / 1.00

En los relojes lógicos de Lamport se cumple que si el evento a ocurre antes que b ($a \rightarrow b$), entonces $C(a) < C(b)$. Sin embargo, observar que $C(x) < C(y)$ NO permite concluir que

$x \rightarrow y$. ¿Por qué?

Seleccione una alternativa:

- a. Porque los relojes de Lamport no son monótonos crecientes.
- b. Porque Lamport exige relojes físicos perfectamente sincronizados, es decir, que deben dar los mismos valores en los mismos tiempos.
- c. Porque $C(x) < C(y)$ siempre implica necesariamente $x \rightarrow y$, para la sincronización de los procesos.
- ✓ d. **Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.**

La respuesta correcta es: Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.

PREGUNTA 14

Finalizado

1.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que arriesgarse a que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

Seleccione una alternativa:

- a. Disponibilidad y tolerancia a particiones (AP).
- b. Las tres propiedades simultáneamente.
- c. Consistencia y disponibilidad (CA).
- ✓ d. **Consistencia y tolerancia a particiones (CP).**

La respuesta correcta es: Consistencia y tolerancia a particiones (CP).

PREGUNTA 15

Finalizado

1.00 / 1.00

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de “me gusta” en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

- ✓ a. **Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.**

- b. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después.
- c. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.
- d. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada.

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

PREGUNTA 16

Finalizado

1.00 / 1.00

Un usuario inicia sesión correctamente con su contraseña, pero el sistema le impide eliminar un archivo que pertenece a otro usuario. ¿Qué mecanismos de seguridad actúan en cada momento?

Seleccione una alternativa:

- a. Autorización en ambos casos.
- ✓ b. **Autenticación al iniciar sesión y autorización al denegar la eliminación.**
- c. Cifrado al iniciar sesión y autenticación al denegar la eliminación.
- d. Autenticación en ambos casos.

La respuesta correcta es: Autenticación al iniciar sesión y autorización al denegar la eliminación.

PREGUNTA 17

Finalizado

0.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes “HOLA” y “MUNDO”, pero en el receptor a veces llegan juntos como “HOLAMUNDO” en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ✗ a. **El sistema cambió automáticamente a UDP.**
- b. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente.
- c. TCP perdió y luego retransmitió un mensaje.
- d. Es un error físico de la tarjeta de red.

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente.

PREGUNTA 18

Finalizado

0.00 / 1.00

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

Enunciado	Respuesta marcada
Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.	“Lee Tus Propias Escrituras” (Read-Your-Writes)
El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.	Consistencia Eventual
Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.	Tolerancia A Fallos Bizantinos
Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.	Tolerancia A Fallos Bizantinos

La respuesta correcta es: Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato. → Consistencia fuerte (linealizable) , Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras. → Consistencia eventual , El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado. → Quórum de mayoría , Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso. → “Lee tus propias escrituras” (read-your-writes).

PREGUNTA 19

Finalizado

1.00 / 1.00

Un equipo intenta mejorar el rendimiento de una aplicación que envía miles de mensajes muy pequeños entre dos nodos. Duplica el ancho de banda del enlace, pero el rendimiento prácticamente no cambia. ¿Qué falacia de la computación distribuida explica este resultado?

Seleccione una alternativa:

- a. “El ancho de banda es infinito”.
- ✓ b. “La latencia es cero”.
- c. “La red es confiable”.
- d. “El transporte es gratuito”.

La respuesta correcta es: “La latencia es cero”.

PREGUNTA 20

Finalizado

0.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- a. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones.
- b. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda.
- ✗ c. **Cliente-servidor de tres capas, porque separa la lógica de negocio.**
- d. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

NAVEGACIÓN

Correcto Parcial Incorrecto

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
17 18 19 20